

Programming For

Problem Solving

UNIT → 1

• Components of computer

- Input unit
- CPU (Central Processing Unit)
- GPU (Graphical Processing Unit)
- PSU (Power supply unit)
- MU (Memory unit) → stores internal memory
- Storage unit → stores external memory
- Output unit
- Mother Board → Assembles all physical components

• IPOS

Input → Process → Output → Storage

Computer works on principle of IPOS

- Input Unit (mouse, keyboard, scanner)
Receives data for computer. data that is processed is put via input unit

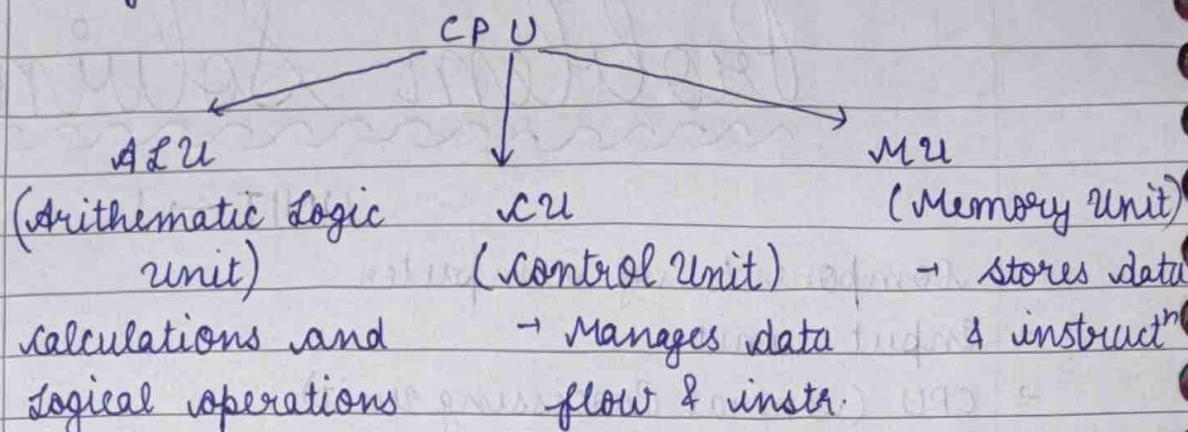
→ computer accepts raw data in binary form

→ Functions

- Takes data to be processed
- Converts raw data into machine readable form
- Transmit converted data into main memory
- Connects user and computer

• CPU

- Brain of computer : controls all the tasks
- Executes instructions, performs calculations, manages data flow



→ ALU

- data inserted via input unit to primary memory
- Performs $+$, $-$, $\times$, $\div$ and calculations [Arithmetics]
- Performs AND, OR, $=$, $<$, $>$ [Logicals]
- Merging, sorting & selection of data
- Sends back data to storage

→ CU

- controller of all activities
- MU send instructions to CU, it converts those instructions which are converted to control signals
- Prioritises & schedules activities
- co-ordinates tasks & syncs input & output units

→ MU [eg → Register] [some data ni jata sirf address jata hai]

- data to be processed or processed data is stored in MU
- It acts as hub of all data & transmits to the computer when required.
- Faster accessing & processing of data.

→ GPU

- Processes images and videos, renders graphics in the applications like gaming, editing, 3D rendering.
- Can be integrated into CPU or exist separately

→ PSU

- Converts electrical power from outlet into usable form for internal components.
- Provides stable and regulated energy
- Prevents damages.

→ Memory (R.A.M → Random Access Memory)

- RAM is computer's short term memory
- Data/Program currently in use is stored temporarily for access by CPU.
- It's volatile i.e. power off, data lost.

→ Storage Devices

- Holds data permanently / for long periods.
- eg → Hard drives, solid state drive
- Retains files even when computer turns off.

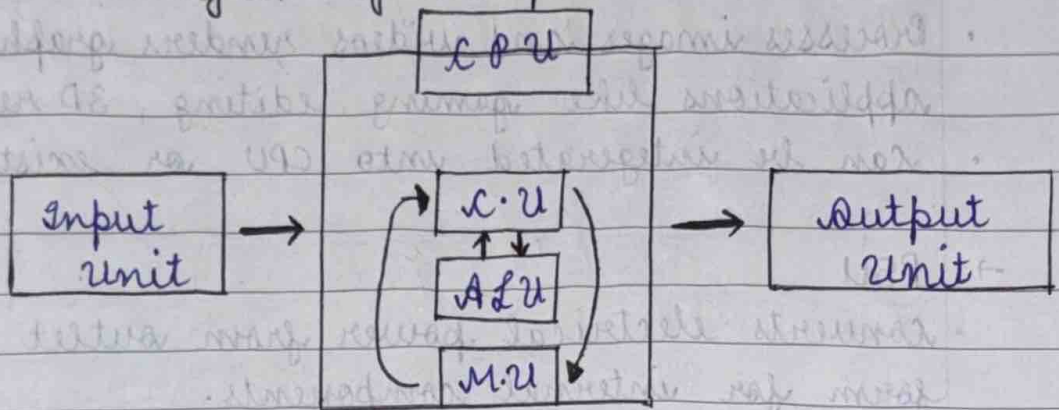
→ Mother Board

- It's the main circuit board connecting all components like CPU, memory, storage & peripheral devices
- Allows communication among all parts.

→ Output Units

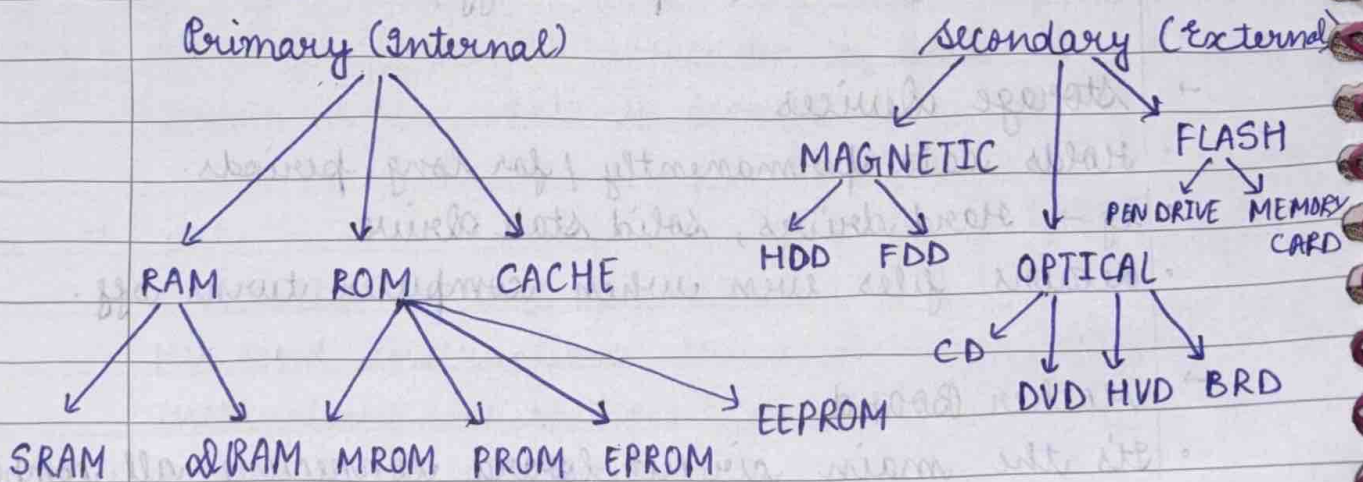
- Communicates information & results back to user
- eg → monitor, printer, speakers.

→ Block diagram of computer



Memory

used for processing and storage



→ Memory

- Electronic storage space where computer keeps the instructions and data it needs to access quickly
- Critical role in computer functioning
- Allow CPU to retrieve & process data efficiently

→ System Bus

- The main channel for communication between CPU and memory bus.

- 1) Data Bus → Transfers actual data between CPU & memory
- 2) Address Bus → Carries memory address that specifies where data should be read from or written to.
- 3) Control Bus → Sends signals to control & co-ordinate activities like read/write operations.

→ Primary Memory (volatile : Power off $\Rightarrow$ data gone)

- Main memory that CPU access directly for the immediate use during active processing.

→ Random Access Memory (RAM)

- Volatile Memory
- Temporarily stores programmes/data.

• DRAM (Dynamic RAM) → To sap presently use kr rhe

- Needs to be refreshed thousands of times per second
- Slower but cost effective.

• SRAM (Static RAM)

- To sapne recently use kra rha frequently use krte ho

- Faster & more reliable than DRAM

- Expensive

- Doesn't need to be refreshed often.

→ ROM (Read Only Memory)

- stores rarely changed software
- non volatile
- stores info even without power
- used to boot or start the computer

• MROM (Maskable ROM) → ~~ek bar khrab~~, can't be reused

→ Hard wired device

→ Pre programmed

→ Low cost

• PROM (Programmable ROM) → reprogrammed but 1 bar

→ Once modifiable

→ Buy blank PROM, program it as required

→ Content can't be erased once written

• EPROM (Erasable Programmable ROM) → ^{Bhut bar ke sath} but time lega

→ Extension of PROM

→ Erase program by exposing to UV rays for 40 min

→ Time consuming

• EEPROM (Electrically Erasable Programmable ROM) → ^{Bhut bar in milisek}

→ Erased electrically

→ Delete & reprogram upto 10,000 times

→ Takes only 4-10 milliseconds

→ Any area can be wiped & programmed selectively

→ CACHE MEMORY

- Processes frequently / recently used data
- Very close to CPU
- Improves speed & efficiency
- Faster than RAM smaller in size
- CPU needs data, first searches in cache

→ Order of speed

Register > Cache > RAM > Secondary Memory
CPU se closeness ↑ Speed ↑ capacity ↓

→ Order of capacity

Secondary > RAM > Cache > Register

→ SECONDARY MEMORY

- Permanent, non-volatile
- Slower but larger capacity

• MAGNETIC STORAGE

→ HDD Hard disc drive

- Spinning disk with magnetic coating
- Large capacity
- Low cost

→ Magnetic Tapes

- Used for back up & archival
- Slower but cheap for large data

• SSD (Flash memory) SOLID State drive

- Use electric current with no moving part

→ USB → Portable, fast, widely used

- expensive, reliable than HDD, increasingly used.

→ OPTICAL STORAGE

- use lasers to read/write

→ CD (compact disc)

700mb storage music, software etc.

→ DVD (digital versatile disc)

4.7 GB, video / software distribution

→ BRD (Blue ray discs)

25 GB+ HD movies & data storage

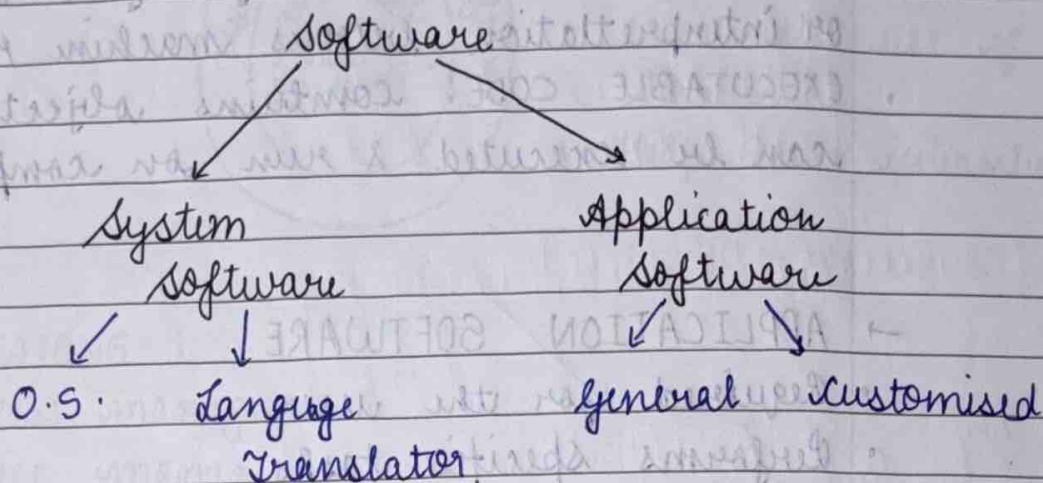
→ HVD

Holographic Versatile disc

3.9 TB

SOFTWARE

It is the collection of programs & program is collection of instructions that instruct computer what to do



→ SYSTEM SOFTWARE

- Manages fundamental operations & hardware
- Acts as bridge b/t user & hardware
- Runs in background
- Platform for app. software
- Usually pre installed

→ OPERATING SYSTEM

- Operates computer
- eg - WINDOWS, IOS, ANDROID, LINUX, MS DOS, UNIX

GUI
GRAPHICAL USER
INTERFACE

Aldest Safest
↑ ↑
CUI
COMMAND USER
INTERFACE

uses icons to perform
action

↓
write a
command to use
& perform actⁿ

→ LANGUAGE TRANSLATOR

- Translates language for computer
- SOURCE CODE: Human readable instructions in programming languages
- OBJECT CODE: source code after compilation or interpretation becomes machine readable obj code
- EXECUTABLE CODE: contains object code and can be executed & run on computer (.exe)

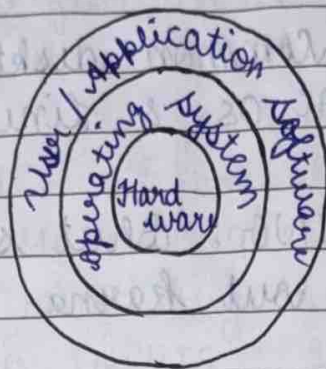
→ APPLICATION SOFTWARE

- Required for the user
- Performs specific tasks
- Known as productivity software
- eg → MS WORD, EXCEL, web browser, Media Player
- User friendly, installed as per needs
- dependent on system software
- GENERAL PURPOSE
- Used commonly
- eg → web browsers, MS OFFICE
- CUSTOMISED PURPOSE
- Customised for specific use
- eg → CRM, HMS, ERP

→ FREEWARE & SHAREWARE SOFTWARE

- | | |
|---|--|
| ↓ | ↓ |
| <ul style="list-style-type: none">• Open for all• Anyone may use• eg - Whatsapp | <ul style="list-style-type: none">• Some free trial then paid• Requires purchasing• eg - Antivirus |

OPERATING SYSTEM



- Fundamental software acts as link between hardware & user
- coordinates use of hardware
- Helps user interaction

→ FUNCTIONS

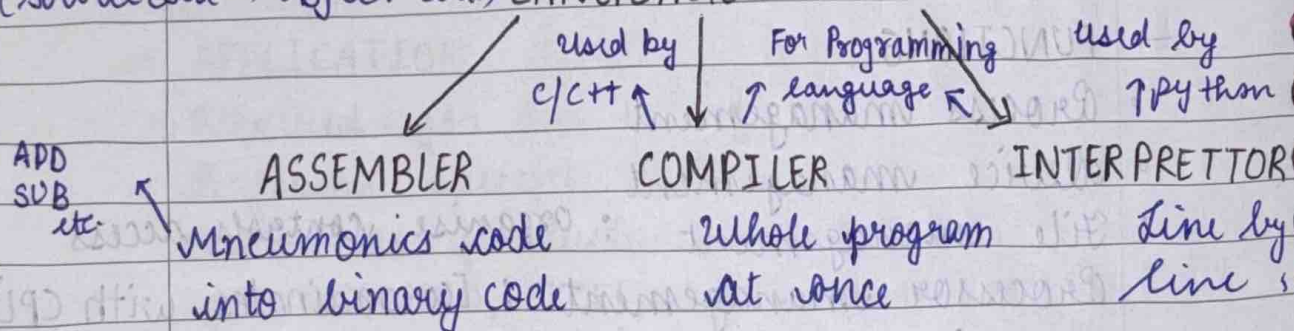
- Process management
- Device management
- File management : organise, controls access
- Processor management : [co-ordinates with CPU]
- Security management : controls authentication & authorisation
- Disk management

→ TYPES OF OS

- BATCH OS : execute batches of jobs without user interaction. eg → Printer (Bhut sari cheejin Broadcast ek sath)
- TIME SHARING : Multiple user interaction by distributing CPU time.
eg → we execute many tasks with single CPU
- DISTRIBUTE : Manages resources across multiple interconnected computers. eg → ek bank me acc khola but kisi bi branch me we get same data
- REAL TIME : Provide intermediate processing & responsible for time critical tasks
It has time limits for time based applications

- **MOBILE OS**: specifically designed for mobiles like android & IOS
- **DESKTOP OS**: common desktop system include microsoft, MAC OS & Linux
- **MULTIPROCESSING**: Bhut sari cheezin ek sath carry out karna.

(Source code → Object code) LANGUAGE TRANSLATOR



→ **COMPILER** pura program ek sath run krke last me pure program ke error ek hi bar me us ap whole batayega.

→ **INTERPRETTOR** ek ek krke line by line errors btayega. because it's better & hence used by PYTHON.

ALGORITHM

It's a set of finite rules / instructions to be followed in calculations or operations

not open to more than 1 interpretation

CHARACTERISTICS OF ALGORITHM

- CLEAR & UNAMBIGUOUS: All instructions must conclude to one single meaning
- WELL DEFINED INPUTS: It should take well defined inputs, may / may not take inputs
- WELL DEFINED OUTPUTS: Clearly define what output should be yielded, should be well def. atleast yielding 1 output.
- FINITENESS: Must be finite and should terminate after a finite time.
- FEASIBLE: Algorithm should be simple, generic, practical, executed with available resources
- LANGUAGE INDEPENDENT: Input in any language should yield the same expected output

REPRESENTATION OF ALGORITHM

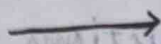
→ Flow chart
→ Pseudo Code

- FLOWCHART
- It's graphical representation,
- easier to understand than natural language
- Used in software development, business process model & engineering

PTO

→ SYMBOLS IN FLOW CHART

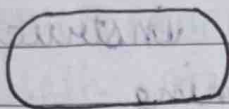
• ARROW



Purpose → Flow line

Indicates direction of flow of logic by connecting symbols

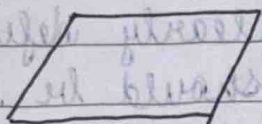
• OVAL



Purpose → Terminal (stop / start)

Indicates the start & end of a flow chart

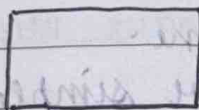
• PARALLELOGRAM



Purpose → Input / output

used for input & output operations

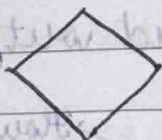
• RECTANGLE



Purpose → Processing

used for arithmetic operations & data manipulations

• DIAMOND



Purpose → Decision

Used for decision making b/t two / more purposes.

→ ALGORITHM TO ADD TWO NUMBERS

STEP 1 START

STEP 2 DECLARE VARIABLES num1, num2 & SUM

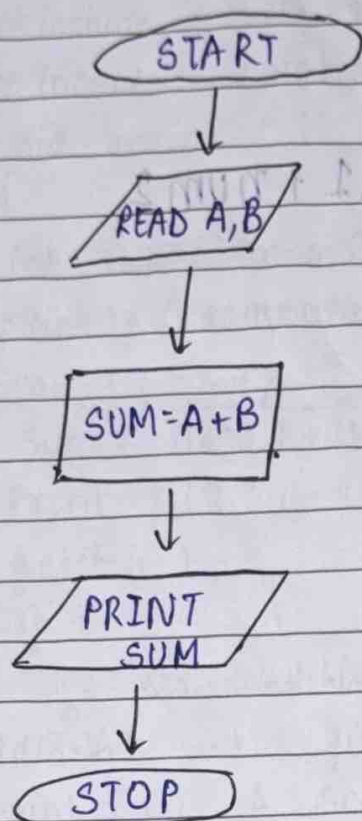
STEP 3 READ VALUES FOR num1, num2

STEP 4 ADD num1, num2 & ASSIGN RESULT TO A VARIABLE SUM.

STEP 5 DISPLAY SUM

STEP 6 STOP

• FLOWCHART REPRESENTATION



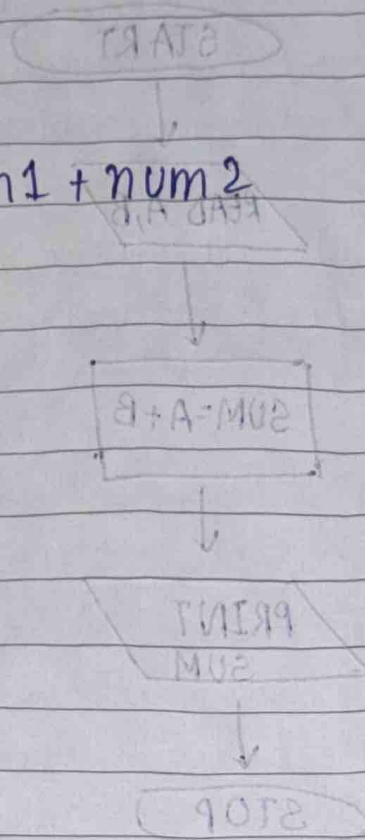
• PSEUDOCODE

- Language independent description of algorithm that combines elements of programming lang. with natural language
- Express in form of annotations & informative text written in plain english which is similar to real code but has no syntax like of any programming languages.
- Can't be compiled / interpreted by computer
- CHARACTERISTICS OF PSEUDOCODE
 - Uses keywords like BEGIN, END, READ, PRINT, IF-THEN-ELSE, FOR, WHILE
 - Emphasise readability / clarity
 - Indentation is used to show structure & hierarchy.

→ PSEUDOCODE TO ADD

```

BEGIN
  READ num 1
  READ num 2
  CALCULATE SUM = num1 + num2
  PRINT SUM
  END
  
```



• PSEUDOCODE

• Language independent description of algorithm that resembles elements of programming language

with natural language

• Express in form of annotations & information that written in plain english which is similar to real code but has no syntactic like of any programming languages.

• Can be compiled / interpreted by computer

• CHARACTERISTICS OF PSEUDOCODE

→ Uses keywords like BEGIN, END, READ, PRINT

IF, THEN, ELSE, FOR, WHILE

→ Emphasize readability / clarity

→ Abstraction is used to show structure & hierarchy

•

→ STRUCTURE OF PROGRAM

#include <stdio.h> } Header File

#include <conio.h>

void main () } Program's execution always start with main ()

Starts code for main function

{
int num 1, num 2, sum; } variable declaration

printf ("enter the value of num 1 & num 2"); } Function

scanf ("%d %d", &num 1, &num 2); } Input function

← sum = num 1 + num 2; } Address operator

printf ("sum = %d", sum);

getch ();

}

standard input output header file

→ <stdio.h> is a file compiler with commands of printf () & scanf (), it saves time

→ <conio.h> has commands of getch () & clrscr ()

↳ console input-output header file

→ printf () → Prints output, as it is on screen

→ scanf () → Takes input from user

→ getchar () → Reads a single character

→ putchar () → Writes a single character

→ fopen () → Opens a file

→ fclose () → closes a file

→ main ()

• It starts execution

• Runs program code

• Return exit status to operating system

→ int main ()

• Returns integer value to O.S.

→ void main()

- Doesn't return any value
- Returns nothing to O.S.

→ VARIABLE DECLARATION

It's a named memory location that is used to store data during program execution. It tells

- name of variable
- type of variable/data it stores

→ VALID VARIABLE NAMES

- Must start with a letter (A-Z or a-z) or underscore
- Can contain letters, digits (0-9) & underscores
- Can't use keywords (int, return etc)
- Case sensitive (Age ≠ age)

→ printf()

- displays output on screen

→ scanf()

- Takes input via keyboards
- Reads formatted inputs & stores values in variable

→ Format Specifiers

- specifies type of data to read

Format	Data Type	Example
%d	Integer	int a = 10;
%f	Float	float pi = 3.14;
%c	character	char ch = 'A';
%s	string	"Hello"
%lf	double	double d = 3.14159;

→ ERROR

It is a discrepancy that can cause program to malfunction leading compiling failures, halting executions, or generating incorrect results.

→ no output shown

→ SYNTAX ERROR → compile time error

- Mistake in typing code's syntax
- Detected by compilers
- Error in structure / grammar of code violates rule of programming
- Program can't run until syntax corrected

For eg :-

```
# include <stdio.h>
```

```
int main () {
```

```
print f("Hello World");
```

```
return 0;
```

```
}
```

} missing closing parenthesis & semicolon

→ SEMANTIC ERROR → wrong output shown

- Grammatically correct but wrong logic
- Gives wrong output
- Code compiles & runs
- eg :-

```
# include <stdio.h>;
```

```
int main() {
```

```
int a=10, b=20;
```

```
int avg = a+b/2
```

```
print f("Average = %.d\n", average);
```

```
return 0;
```

```
}
```

} Wrong logic $\div$ before +

$(a+b)$

$\frac{2}{a+b}$

but

$a+b$

$\frac{2}{a+b}$ like h

→ RUNTIME ERROR

- During execution after compilation
- Trick to identify; not found by compiler
- Includes division by 0, array index out of bound & memory leaks
- eg:-

```
#include <stdio.h>
int main () {
    int a=10, b=0;
    int result = a/b; // division by 0
    printf("Result = %d/n", result);
    return 0;
}
```

Undefined Behaviour

- SEMANTIC ERROR → Wrong output shown
- Grammatically correct but wrong logic
- Prints wrong output
- Code compiles & runs
- eg:-

```
#include <stdio.h>
int main () {
```

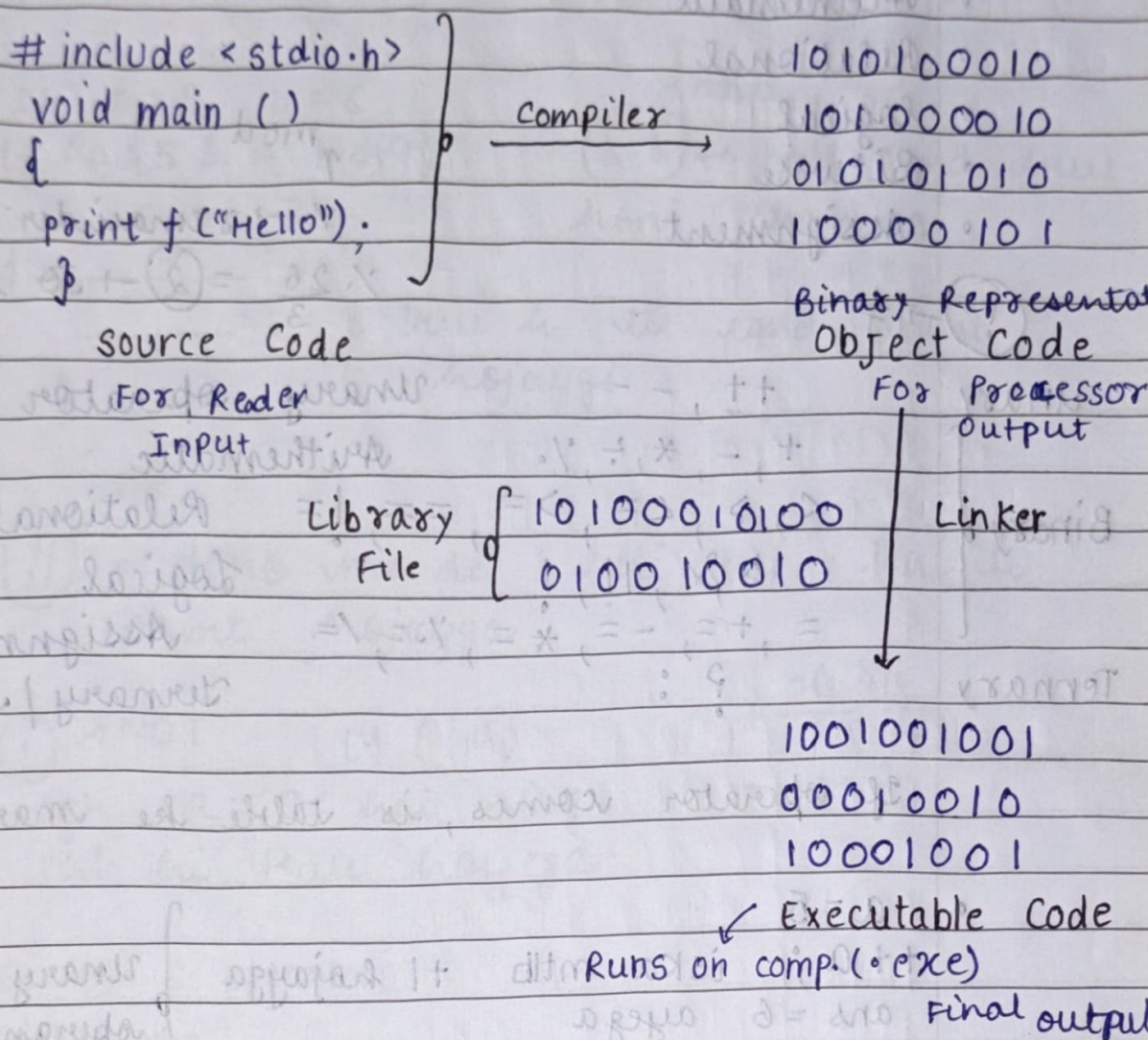
```
    int a=10, b=20;
```

```
    int avg = (a+b)/2;
```

```
    printf("Average = %d/n", avg);
```

```
    return 0;
}
```


→ Source code, object code & executable code



→ Obj code isn't directly executable

→ Executable code uses library files to resolve external references to produce complete standalone program.